

TÉCNICAS DE PROGRAMAÇÃO 1

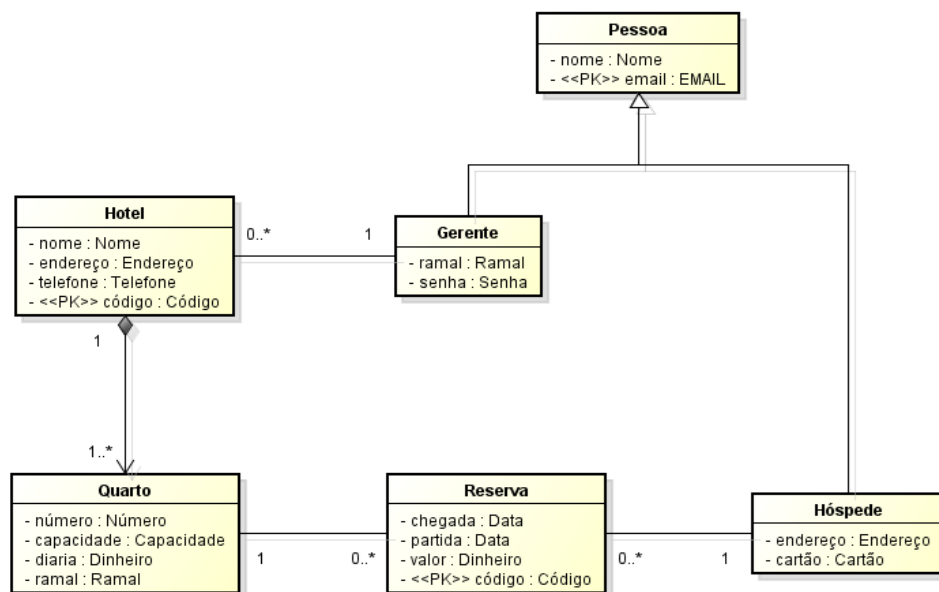
ESPECIFICAÇÃO DO TRABALHO PRÁTICO

1. ATIVIDADES A REALIZAR

1. Projetar, codificar e documentar classes domínio (*domain classes*).
2. Projetar, codificar e documentar classes entidade (*entity classes*).
3. Construir modelo de arquitetura do software, codificar e documentar as interfaces entre os módulos.
4. Projetar e codificar a camada de apresentação.
5. Projetar e codificar a camada de serviço.
6. Criar vídeo que demonstre a execução com sucesso de teste fumaça (*smoke test*) do código integrado.

2. REQUISITOS FUNCIONAIS DO SISTEMA DE SOFTWARE

O sistema de software visa facilitar a gestão de pequenos hotéis. Para usar o sistema, o gerente deve criar uma conta. Após criar uma conta, deve ser autenticado informando o seu email e a sua senha. Após ser autenticado, o gerente pode acessar os serviços disponibilizados pelo sistema. A seguir, tem-se uma relação desses serviços: ler, editar e excluir conta; criar, ler, editar e excluir hotel; criar, ler, editar e excluir quarto; criar, ler, editar e excluir reserva; criar, ler, editar e excluir hóspede; listar hotéis (apresenta dados de cada hotel); listar quartos (apresenta dados de cada quarto); listar reservas (apresenta dados de cada reserva); listar hóspedes (apresenta dados de cada hóspede). Não podem ocorrer conflitos entre reservas. Não é possível editar dado que identifica registro (chave primária). Finalmente, o sistema deve assegurar as multiplicidades informadas nos relacionamentos presentes no seguinte diagrama de classes e também não deve possibilitar exclusões que resultem em inconsistências.



DOMÍNIOS	
NOME	FORMATO VÁLIDO
Capacidade	1, 2, 3 ou 4
Cartão	16 dígitos Presença de dígito verificador calculado segundo o algoritmo de Luhn
Código	10 caracteres Cada carcter é letra (a-z) ou dígito (0-9)
Data	dia-mês-ano Dia é número de 1 a 31 Mês é JAN, FEV, MAR, ABR, MAI, JUN, JUL, AGO, SET, OUT, NOV ou DEZ

	Ano é número de 2000 a 2999. Data deve ser válida considerando os anos bissextos.
Dinheiro	0,01 a 1.000.000,00 Armazenamento de ser em inteiro para evitar erro de arredondamento.
EMAIL	parte-local@domínio Parte local pode conter letra (a-z), dígito (0-9) ou ponto(.) ou hífen (-); não pode iniciar ou terminar com ponto ou hífen; ponto ou hífen deve ser seguido por letra(s) ou dígito(s); comprimento máximo é 64 caracteres. Domínio é composto por uma ou mais partes separadas por ponto (.); cada parte pode conter letra (a-z), dígito (0-9) ou hífen (-); não pode iniciar ou terminar com hífen; comprimento máximo é 255 caracteres.
Endereço	Texto com 5 a 30 caracteres. Caracter pode ser letra (a-z ou A-Z), dígito (0-9), vírgula, ponto ou espaço em branco; vírgula não pode ser seguida por vírgula ou ponto; ponto não pode ser seguido por vírgula ou ponto; espaço em branco é seguido por letra ou dígito; primeiro e último caracter não pode ser vírgula, ponto ou espaço em branco.
Nome	Texto com 5 a 20 caracteres. Cada caracter pode ser letra (a-z ou A-Z) ou espaço em branco; primeiro caracter de cada termo é letra maiúscula (A-Z); espaço em branco é seguido por letra; último caracter não pode ser espaço em branco.
Número	001 a 999
Ramal	00 a 50
Senha	5 caracteres Caracter pode ser letra (a-z ou A-Z), dígito (0-9) ou caracter especial (! " # \$ % & ?); letra não pode ser seguida por letra, dígito não pode ser seguido por dígito; existe pelo menos uma letra minúscula (a-z), uma letra maiúscula (A-Z), um dígito (0-9) e um caracter especial.
Telefone	+DDDDDDDDDDDDDDDD D é dígito (0-9)

3. REQUISITOS NÃO FUNCIONAIS DO SISTEMA DE SOFTWARE

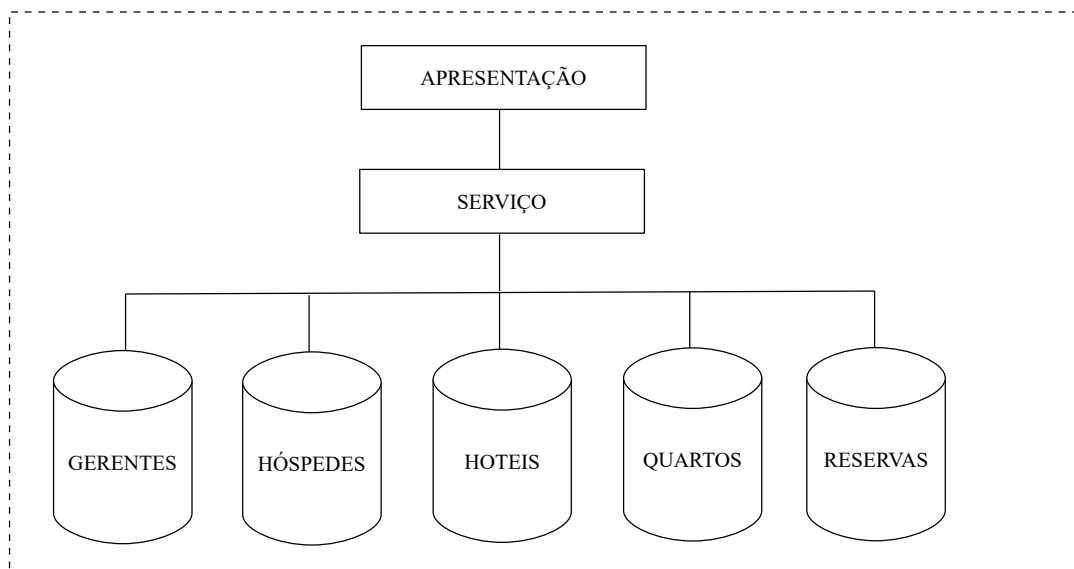
1. Adotar o estilo de arquitetura em camadas (*layers*).
2. A arquitetura do software deve ser composta por camada de apresentação e por camada de serviço.
3. A camada de apresentação deve ser responsável pela interface com o usuário e pela validação dos dados de entrada.
4. A camada de serviço deve ser responsável pela lógica de negócio e por armazenar dados.
5. Cada camada deve ser decomposta em módulos de software.
6. Módulos de software devem interagir por meio de serviços especificados em interfaces.
7. Módulos de software devem ser decompostos em classes.
8. A arquitetura do software deve ser composta por ao menos quatro módulos.
9. Devem ser implementadas classes que representem domínios, entidades, controladoras e contêineres.
10. Implementar o código na linguagem de programação C++.
11. Prover projeto compatível com o ambiente de desenvolvimento Code::Blocks.
12. Nas implementações dos códigos de validação não é necessário considerar acentuação e nem letra ç.

4. REQUISITOS DO TRABALHO

1. Trabalho pode ser realizado individualmente ou por equipe com até quatro participantes.
2. Preencher os documentos com clareza, atentar para a ortografia e adotar um padrão de codificação (*coding standard*).
3. Prover arquivo com identificação de cada membro da equipe e identificações de artefatos que cada membro contruiu.
4. Desenvolver o sistema de software segundo os requisitos funcionais e não funcionais especificados.
5. Cada classe domínio deve conter atributo que seja instância de tipo suportado pela linguagem de programação.
6. Cada classe domínio deve permitir acesso ao atributo por meio de métodos públicos *set* e *get*.
7. Método *set* de cada classe domínio deve lançar exceção em caso de formato inválido.
8. Cada classe entidade deve conter atributos onde cada atributo é instância de classe domínio.
9. Cada classe entidade deve permitir acesso aos atributos por meio de métodos públicos *set* e *get*.
10. Modelo de arquitetura deve conter diagrama composto por módulos, interfaces entre módulos e relacionamentos.

11. Diagrama integrante do modelo de arquitetura deve ser construído usando a linguagem de modelagem UML.
12. Modelo de arquitetura deve conter descrições textuais das responsabilidades de cada módulo.
13. Fornecer modelo de arquitetura de software em arquivo PDF.
14. Declarar em código as interfaces entre módulos por meio de classes abstratas.
15. Classes abstratas devem ser compostas por métodos virtuais puros.
16. Documentar cada classe (domínios, entidades, interfaces etc.) em HTML usando a ferramenta Doxygen.
17. Documentar as classes segundo perspectiva dos usuários das classes.
18. Fornecer as páginas HTML geradas pela ferramenta Doxygen.
19. Camada de apresentação deve depender de serviços relacionados em interfaces declaradas.
20. Camada de apresentação pode ser codificada usando *cin* e *cout*, *PDCurses* ou interface gráfica.
21. Entre as classes da camada de apresentação devem existir classes controladoras.
22. Camada de serviço deve implementar serviços relacionados em interfaces declaradas.
23. Camada de serviço pode armazenar os objetos em estrutura de dados em memória (fila, pilha etc.).
24. Camada de serviço pode armazenar os objetos em banco de dados relacional usando o produto SQLite.
25. Entre as classes da camada de serviço devem existir classes controladoras e classes contêineres.
26. Fornecer vídeo que demonstre teste fumaça (*smoke test*) e a execução com sucesso do código integrado.
27. No teste fumaça (*smoke test*) deve haver um caso de teste para cada funcionalidade presente nos requisitos funcionais.
28. Cada caso de teste deve contemplar um cenário de sucesso.
29. Fornecer projeto Code::Blocks que possibilite compilar e executar os códigos sem erros na plataforma de correção.
30. Fornecer os códigos em formato fonte e em formato executável.
31. Incluir todos os artefatos construídos em um arquivo zip e atribuir o nome T1-TP1-X-Y-Z.ZIP ao arquivo zip.
32. No nome do arquivo zip, X, Y e Z devem ser os números de matrícula dos autores do trabalho.
33. Testar se o arquivo pode ser descompactado com sucesso e se não há vírus no arquivo.
34. Enviar o arquivo dentro do prazo.
35. Não cumprimento de requisitos reduz a nota do trabalho.

5. ELEMENTOS DO SISTEMA DE SOFTWARE



6. CRITÉRIOS DE AVALIAÇÃO

ITEM	CRITÉRIO	% ACERTO
1	• Cada domínio é uma classe. • Cada classe contém métodos set e get públicos. • Cada método set lança exceção se formato de valor de parâmetro é inválido. • Cada classe documentada por texto que a descreve. • Cada membro público da classe (atributo ou método) documentado por texto que o descreve. • Provida documentação em formato HTML.	0, 25, 50, 75, 100
2	• Cada entidade é uma classe. • Cada classe contém atributos que são instâncias de domínios. • Cada atributo da classe é acessível por métodos set e get públicos. • Cada classe documentada por texto que a descreve. • Cada membro público da classe (atributo ou método) documentado por texto que o descreve. • Provida documentação em formato HTML.	0, 25, 50, 75, 100
3	• Diagrama integrante do modelo de arquitetura é composto por módulos, interfaces e relacionamentos. • Linguagem UML usada no diagrama integrante do modelo de arquitetura. • Provida descrição textual do propósito de cada módulo. • Provida declaração de cada interface por meio de classe abstrata. • Cada classe documentada por texto que a descreve. • Cada membro público da classe (atributo ou método) documentado por texto que o descreve. • Provida documentação em formato HTML.	0, 25, 50, 75, 100
4	• Providas classes contendo código responsável por interface com o usuário e validação de dados de entrada. • Classes dependem de interfaces que especificam serviços providos pela camada de negócio. • Cada classe documentada por texto que a descreve. • Cada membro público da classe (atributo ou método) documentado por texto que o descreve. • Provida documentação em formato HTML.	0, 25, 50, 75, 100
5	• Providas classes contendo código responsável por lógica de negócio e armazenamento de dados. • Classes realizam interfaces que especificam serviços providos pela camada de negócio. • Cada classe documentada por texto que a descreve. • Cada membro público da classe (atributo ou método) documentado por texto que o descreve. • Provida documentação em formato HTML.	0, 25, 50, 75, 100
6	• Fornecido vídeo que demonstra teste fumaça (<i>smoke test</i>). • Teste fumaça é composto por caso de teste de sucesso para cada funcionalidade provida pelo sistema. • Teste fumaça executa com sucesso o código integrado do sistema.	0, 25, 50, 75, 100